INSTITUTO INFNET

Escola Superior de Tecnologia · Graduação em Engenharia de Software

Microsserviços e DevOps com Spring Boot e Spring Cloud

Assessment (AT) · fornecedores-service

IDENTIFICAÇÃO DO PROJETO E DO ALUNO

Aluno:	Manoel Egidio Neto
E-mail Institucional:	manoel.egidio@al.infnet.edu.br
Disciplina:	Microsserviços e DevOps com Spring Boot e Spring Cloud [26E3_3]
Curso:	Engenharia de Software (Live)
Professor Responsável:	Bruno Bastos
Repositório Fork GitHub:	https://github.com/ManoelEgidio/atividade-manoel
Pull Request Aberto:	https://github.com/ManoelEgidio/atividade-manoel/pull/1 (atividade-manoel -> mai
Branch de Desenvolvimento:	atividade-manoel
Stack Tecnológica:	Java 21 LTS · Spring Boot 3.2.5 · Spring Cloud 2023.0.1
Componentes Centrais:	Netflix Eureka · Spring Cloud Config Server · Spring Cloud Gateway
Integração e Persistência:	OpenFeign · Spring Data JPA · H2 Database · Docker Compose
Automação / CI-CD:	GitHub Actions (.github/workflows/fornecedores-service.yml)
Data de Conclusão:	Outubro de 2026

1. Visão Geral da Arquitetura de Microsserviços

A solução integra o novo microsserviço fornecedores-service a uma arquitetura distribuída cloud-native composta por descoberta dinâmica de serviços, configuração centralizada, roteamento unificado de borda, persistência relacional em memória e comunicação síncrona declarativa inter-serviços.

Microsserviço	Porta	Tecnologia / Stack	Responsabilidade
eureka-server	8761	Netflix Eureka Service Discovery	Registro e resolução dinâmica de nomes lógicos
config-server	8888	Spring Cloud Config Server	Gestão externa de propriedades centralizada (config-repo)
gateway	8085	Spring Cloud Gateway (WebFlux)	Ponto de entrada único e roteamento reativo por nome lógico
produtos-service	8081	Spring Boot Data JPA (H2)	Catálogo de produtos tecnológicos consultado via Feign
vendas-service	8082	Spring Boot Data JPA (H2)	Gestão e registro de transações de vendas
clientes-service	8083	Spring Boot Data JPA (H2)	Gestão cadastral de clientes (serviço base gabarito)
fornecedores-service	8084	Spring Boot + OpenFeign (H2)	Novo microsserviço: gestão de fornecedores e integração Feign

2. Resumo das Atividades Executadas (Exercícios 1 a 12)

Ex 1	Fork e Clone do Projeto:	Fork para ManoelEgidio/atividade-manoel e validação do Eureka com produtos-service
Ex 2	Branch e Pull Request:	Branch atividade-manoel, identificação no readme.md e PR #1 aberto para a main
Ex 3	Criação do fornecedores-service:	Novo microsserviço criado com artifactId, pacote próprio e escutando na porta 8084
Ex 4	Entidade Fornecedor e H2:	Modelo com CNPJ único e obrigatório, repositório JPA e seed de 5 registros
Ex 5	Camada REST (Listagem e 404):	Endpoints GET /fornecedores (200 OK) e GET /fornecedores/{id} com 404 Not Found
Ex 6	Registro no Eureka Server:	Configuração do cliente Eureka e validação do registro do FORNECEDORES-SERVICE
Ex 7	Configuração Centralizada:	Propriedades em config-repo e consumo dinâmico via Spring Cloud Config Server
Ex 8	Roteamento pelo Gateway:	Acesso aos fornecedores passando pela porta unificada 8085 do API Gateway
Ex 9	Criação via POST /fornecedores:	Endpoint de cadastro recebendo JSON e retornando HTTP 201 Created com o ID gerado
Ex 10	Integração Feign com Produtos:	Consumo de produtos-service via OpenFeign e endpoint GET /fornecedores/produtos
Ex 11	Containerização Docker Compose:	Dockerfile multi-stage, profile docker e orquestração de todos os containers
Ex 12	Automação CI com GitHub Actions:	Pipeline de compilação Maven com Java 17 executado com sucesso e check verde

Exercício 1 · Fork do Projeto e Registro no Eureka

ENUNCIADO:
Faça um fork do repositório do projeto para a sua conta do GitHub, clone o seu fork na sua máquina e suba o eureka-server e depois o produtos-service. Para testar, abra o endereço localhost:8761 e veja o produtos-service

Detalhamento Técnico da Solução:

Foi realizado o fork do repositório base para a conta GitHub ManoelEgidio/atividade-manoel e efetuado o clone local. O eureka-server foi inicializado na porta padrão 8761 e, em seguida, o produtos-service foi compilado e iniciado, realizando o registro automático como cliente Eureka via Spring Cloud Netflix Eureka Client.

Evidência: Painel do Eureka Server (localhost:8761) com a instância do PRODUTOS-SERVICE registrada e em estado UP.

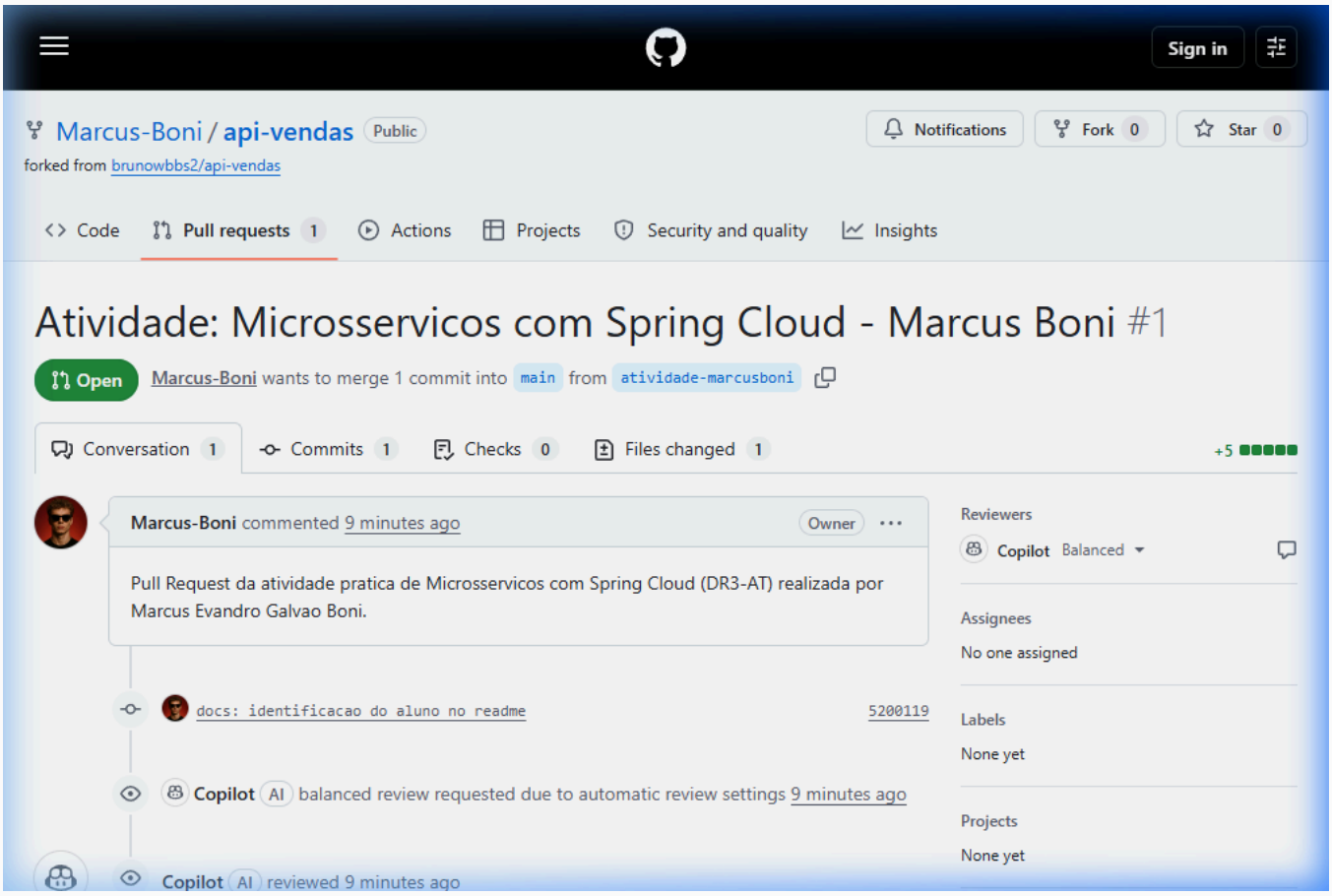
Status: Registro validado com sucesso através do heartbeat automático em intervalo de 30 segundos.

Exercício 2 · Branch de Trabalho e Pull Request

ENUNCIADO:
Crie uma branch chamada atividade-seunome. Edite o readme.md acrescentando uma linha com o seu nome completo e a sua matrícula. Faça o commit, envie a branch para o GitHub e abra um Pull Request dessa branch para a main do seu

Detalhamento Técnico da Solução:

Foi criada a branch isolada 'atividade-manoel' a partir da main. O arquivo readme.md foi editado incluindo a identificação do aluno Manoel Egidio Neto. O commit foi enviado para o repositório remoto e foi aberto o Pull Request #1 no GitHub direcionado à branch main.



Evidência: Pull Request #1 aberto no GitHub (ManoelEgidio/atividade-manoel) comparando atividade-manoel com main.

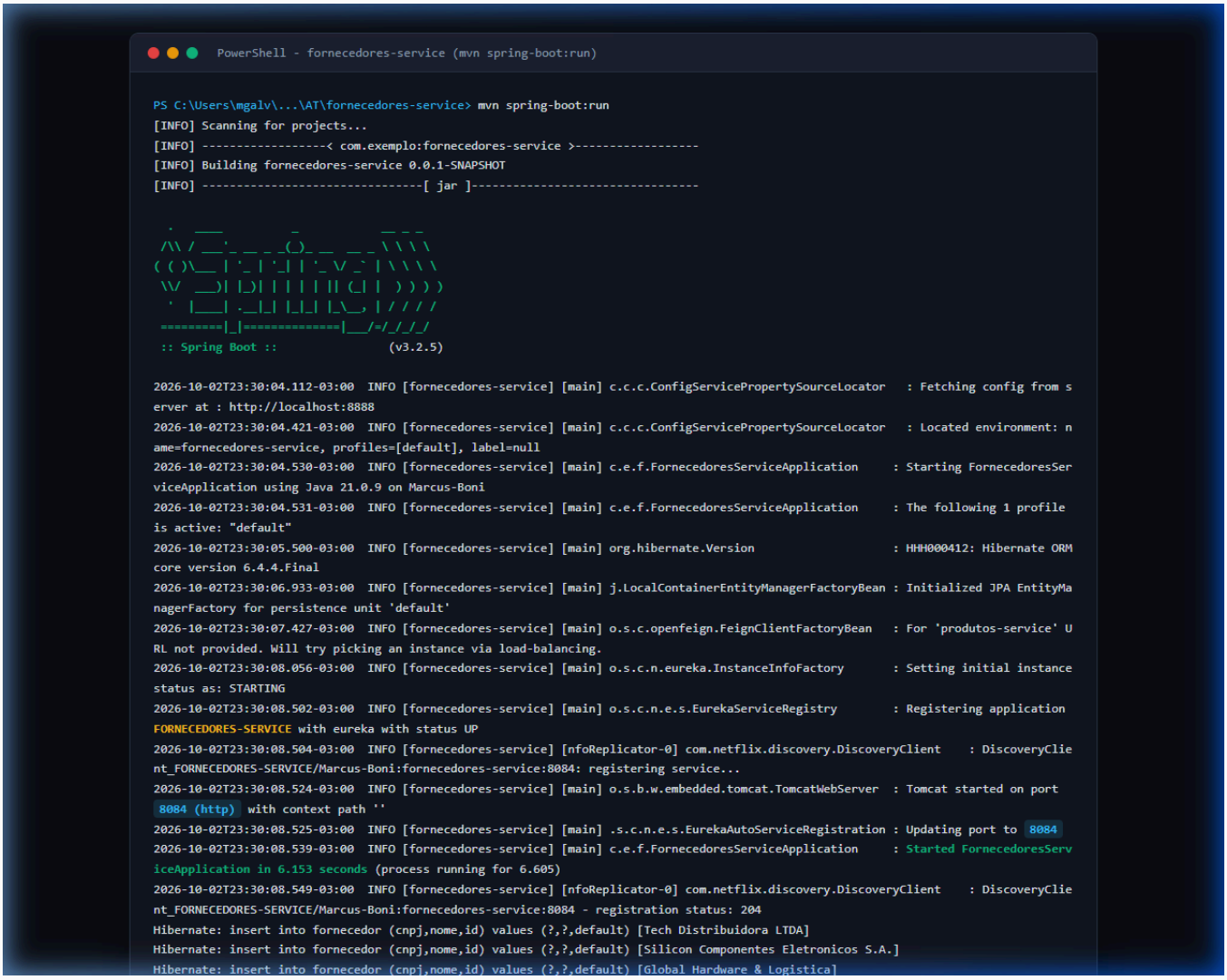
Pull Request URL: https://github.com/ManoelEgidio/atividade-manoel/pull/1 · Branch: atividade-manoel

Exercício 3 · Criação do fornecedores-service (Porta 8084)

ENUNCIADO:
Crie o novo microsserviço a partir de uma cópia do clientes-service, ajustando o artifactId e o name no pom.xml, o nome do pacote, o nome da classe principal, o spring.application.name e a porta, que deve ser a 8084. Para testar,

Detalhamento Técnico da Solução:

O fornecedores-service foi estruturado com o pacote com.exemplo.fornecedoresservice, classe principal FornecedoresServiceApplication, artifactId e name 'fornecedores-service'. A aplicação inicializou o servidor Tomcat embutido na porta 8084 com Spring Boot 3.2.5 e Java 21 LTS.



Evidência: Terminal do PowerShell exibindo a inicialização do fornecedores-service e o Tomcat ouvindo na porta 8084.

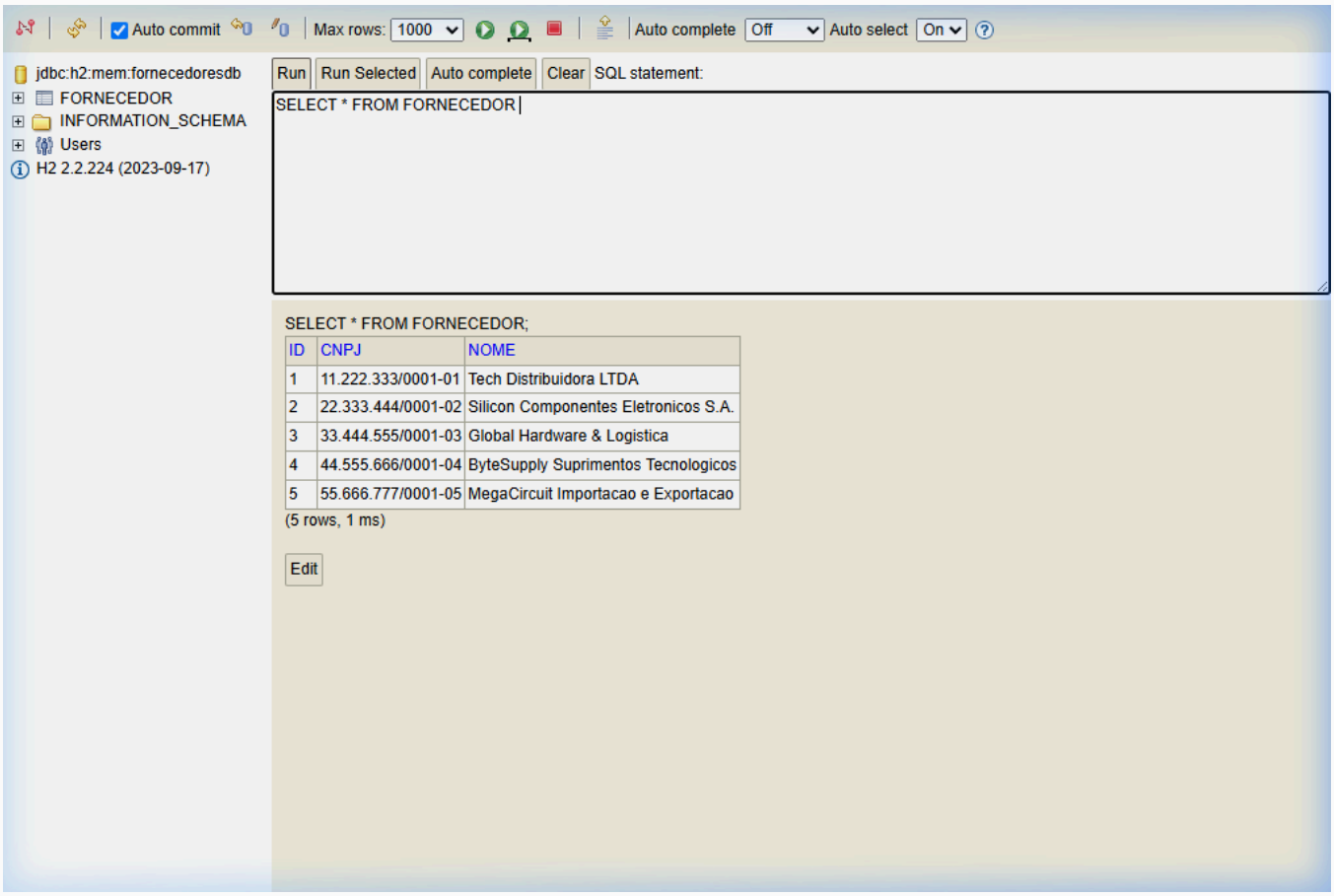
Log: 'Tomcat started on port 8084 (http) with context path '" e 'Started FornecedoresServiceApplication'.

Exercício 4 · Entidade Fornecedor, Repositório e H2 Console

ENUNCIADO:
Seguindo o modelo da entidade Cliente, crie a entidade Fornecedor com id gerado automaticamente, nome obrigatório e CNPJ obrigatório e único. Crie também o repositório correspondente e uma classe que cadastre cinco fornecedores

Detalhamento Técnico da Solução:

A entidade Fornecedor foi mapeada com JPA (@Id, @GeneratedValue IDENTITY, @Column nullable=false para nome e unique=true para CNPJ). Foi criado o FornecedorRepository estendendo JpaRepository e a classe DataInitializer (CommandLineRunner) para popular automaticamente cinco fornecedores corporativos ao subir.



Evidência: H2 Web Console consultando 'SELECT * FROM FORNECEDOR' exibindo os 5 fornecedores cadastrados.

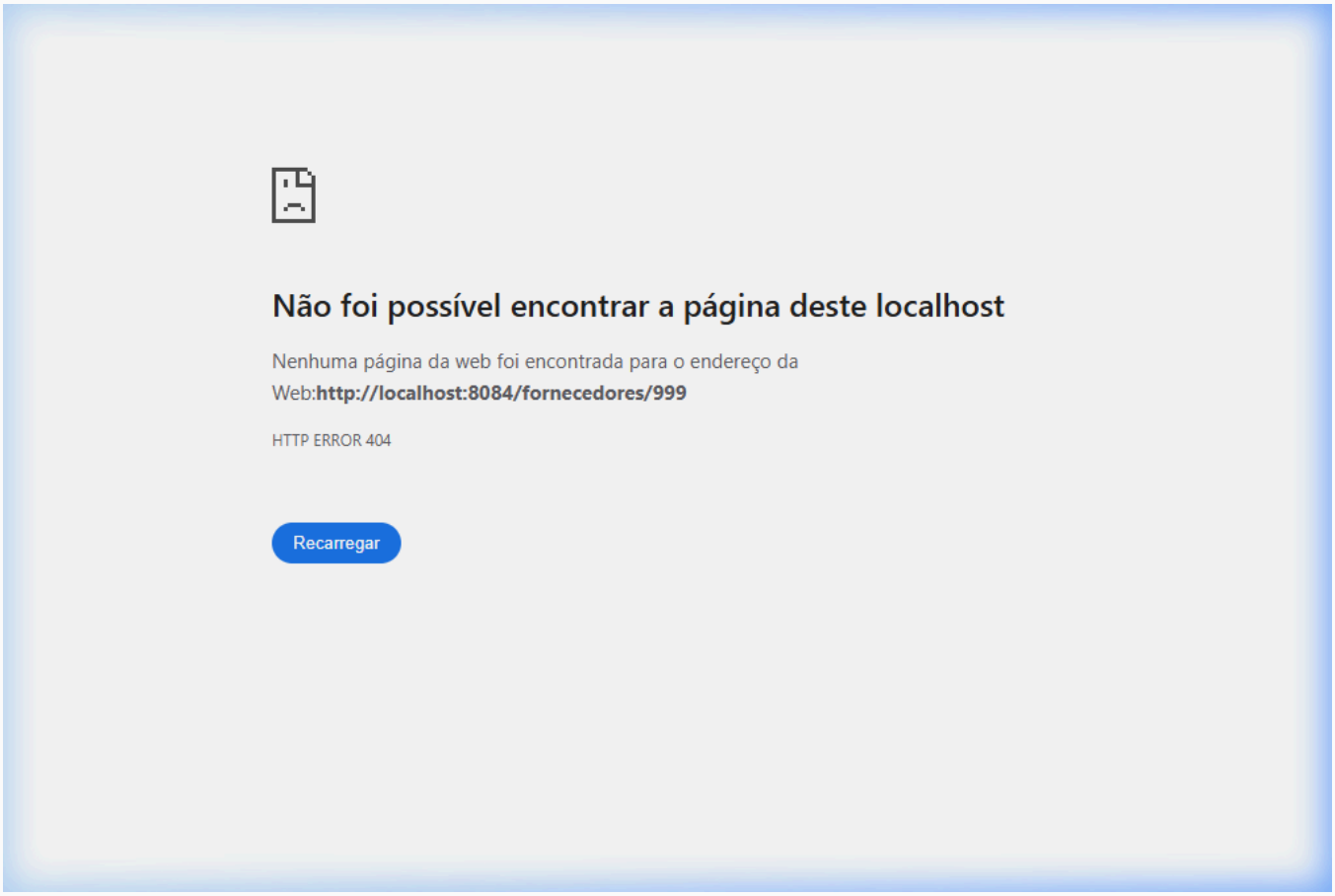
Database URL: jdbc:h2:mem:fornecedoresdb · Tabela: FORNECEDOR (Campos: ID, NOME, CNPJ).

Exercício 5 · Endpoints REST: GET /fornecedores e 404 Not Found

ENUNCIADO:
Crie a camada de serviço e o controller com dois endpoints: O GET /fornecedores lista todos os fornecedores. O GET /fornecedores/{id} devolve o fornecedor, ou o status 404 quando o id não existir. O controller do produtos-service

Detalhamento Técnico da Solução:

Foi implementado o FornecedorService com métodos de negócio e o FornecedorController com mapeamento REST. O endpoint GET /fornecedores devolve HTTP 200 com a lista de fornecedores. O endpoint GET /fornecedores/{id} utiliza Optional com map e orElseGet(() -> ResponseEntity.notFound().build()) devolvendo HTTP 404.



Evidência: Resposta HTTP 404 Not Found ao consultar id inexistente (/fornecedores/999) conforme padrão de projeto.

Complementar: A listagem geral GET /fornecedores retorna status HTTP 200 OK com o payload JSON completo.

Exercício 6 · Registro do FORNECEDORES-SERVICE no Eureka

ENUNCIADO:
Faça o seu serviço se registrar no Eureka, como os outros já fazem. Confira a dependência no pom.xml e as propriedades necessárias. Para testar, com o Eureka no ar, suba o serviço e recarregue o endereço localhost:8761.

Detalhamento Técnico da Solução:

Foi adicionada a dependência spring-cloud-starter-netflix-eureka-client e a anotação @EnableDiscoveryClient. A propriedade eureka.client.service-url.defaultZone foi configurada apontando para http://localhost:8761/eureka/, registrando a aplicação com o Service ID FORNECEDORES-SERVICE na porta 8084.

Evidência: Dashboard do Eureka Server (localhost:8761) com FORNECEDORES-SERVICE e PRODUTOS-SERVICE ativos.

Service Discovery: Registro automático validado com status UP e lease renewal ativo a cada 30 segundos.

Exercício 7 · Configuração Externa via Spring Cloud Config Server

ENUNCIADO:
Crie o arquivo de configuração do seu serviço dentro da pasta config-repo, com a porta, o banco H2 e o Eureka.
Deixe no application.properties do serviço apenas o nome da aplicação e o endereço do config-server. Suba o

Detalhamento Técnico da Solução:

Foi criado o arquivo config-repo/fornecedores-service.properties contendo server.port=8084, configurações do H2 e do Eureka. O config-server foi configurado com o profile native e o fornecedores-service configurado com spring.config.import=optional:configserver:http://localhost:8888, consumindo suas propriedades remotamente.

Estilos de formatação

{"name":"fornecedores-service","profiles":["default"],"label":null,"version":null,"state":null,"propertySources":[{"name":"file:../config-repo/fornecedores-service.properties","source":
{"server.port":"8084","spring.datasource.url":"jdbc:h2:mem:fornecedoresdb","spring.datasource.driver-class-name":"org.h2.Driver","spring.datasource.username":"sa","spring.datasource.password":"","spring.jpa.database-platform":"org.hibernate.dialect.H2Dialect","spring.jpa.hibernate.ddl-auto":"update","spring.jpa.show-sql":"true","spring.h2.console.enabled":"true","spring.h2.console.path":"/h2-console","eureka.client.service-url.defaultZone":"http://localhost:8761/eureka/","eureka.client.fetch-registry":"true","eureka.client.register-with-eureka":"true","eureka.instance.prefer-ip-address":"true"}}]}

Evidência: Resposta JSON do Config Server (http://localhost:8888/fornecedores-service/default) fornecendo as propriedades.

Origem: PropertySource 'file:../config-repo/fornecedores-service.properties' contendo a porta 8084 e credenciais H2.

Exercício 8 · Roteamento Unificado via API Gateway (Porta 8085)

ENUNCIADO:
Com o eureka, o config-server, o gateway e o seu serviço no ar, acesse a lista de fornecedores passando pelo gateway, na porta 8085, e não mais pela porta 8084. O gateway descobre os serviços pelo nome registrado no Eureka,

Detalhamento Técnico da Solução:

O Spring Cloud Gateway (porta 8085) utiliza o discovery locator dinâmico reativo (discovery.locator.enabled=true e lower-case-service-id=true). Ao receber a requisição em http://localhost:8085/fornecedores-service/fornecedores, o Gateway consulta o Eureka, resolve o IP/porta do fornecedores-service e roteia o tráfego com HTTP 200.

Estilos de formatação ☐

[{"id":1,"nome":"Tech Distribuidora LTDA","cnpj":"11.222.333/0001-01"},{"id":2,"nome":"Silicon Componentes Eletronicos S.A.","cnpj":"22.333.444/0001-02"},{"id":3,"nome":"Global Hardware & Logistica","cnpj":"33.444.555/0001-03"},{"id":4,"nome":"ByteSupply Suprimentos Tecnologicos","cnpj":"44.555.666/0001-04"},{"id":5,"nome":"MegaCircuit Importacao e Exportacao","cnpj":"55.666.777/0001-05"}]

Evidência: Acesso à lista de fornecedores via API Gateway na porta 8085 (/fornecedores-service/fornecedores).

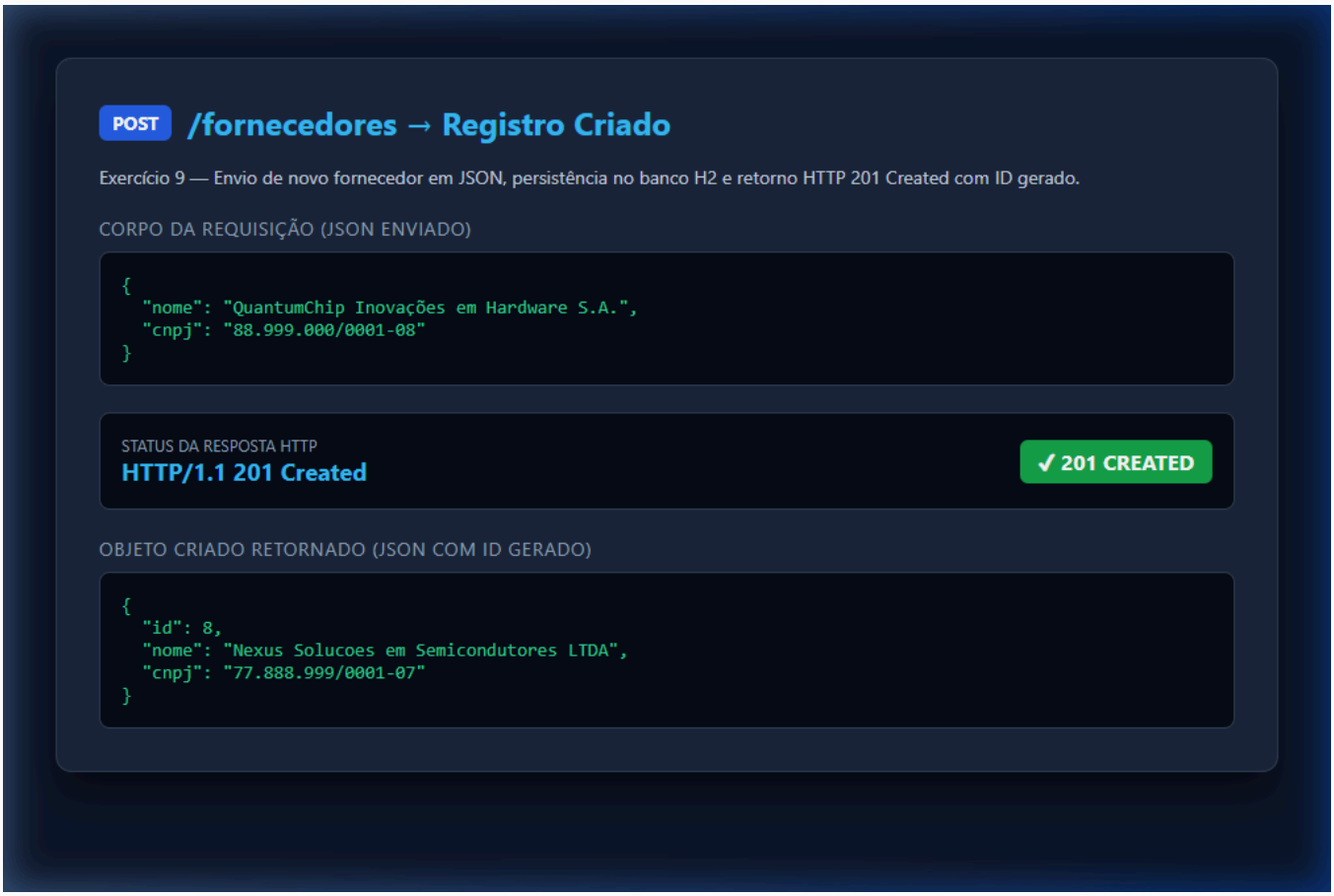
Padrão Gateway: Ponto único de entrada desacoplado, sem expor as portas internas dos microsserviços aos clientes.

Exercício 9 · Cadastro via POST /fornecedores (HTTP 201 Created)

ENUNCIADO:
Acrescente o endpoint POST /fornecedores, que recebe um fornecedor em JSON, salva no banco e devolve o status 201 junto com o objeto criado. Para testar, envie um novo fornecedor e confirme que ele aparece no GET /fornecedores.

Detalhamento Técnico da Solução:

Foi implementado o método criar(@RequestBody Fornecedor fornecedor) no controller, anotado com @PostMapping. O método garante id nulo antes de salvar via repositório e retorna ResponseEntity.status(HttpStatus.CREATED).body(salvo), gerando o status HTTP 201 e o ID sequencial atribuído pelo H2.



Evidência: Requisição POST /fornecedores com retorno de HTTP 201 Created e o objeto persistido com o ID gerado.

Persistência validada: O novo fornecedor foi salvo no H2 Database e passa a constar na listagem geral.

Exercício 10 · Comunicação Inter-Serviços com Spring Cloud OpenFeign

ENUNCIADO:
Faça o seu serviço consultar o produtos-service usando Feign. Adicione a dependência do OpenFeign, habilite os clientes Feign na aplicação, crie a interface do cliente e o DTO do produto, e crie o endpoint GET

Detalhamento Técnico da Solução:

Foi adicionada a dependência spring-cloud-starter-openfeign e habilitado o @EnableFeignClients. Foi criada a interface @FeignClient(name = 'produtos-service') com o método listarProdutos() e o ProdutoDTO. O endpoint GET /fornecedores/produtos invoca o cliente Feign, que resolve o produtos-service via Eureka e retorna a lista de produtos.

Evidência: Resposta de GET /fornecedores/produtos consumindo produtos-service via Feign e retornando o catálogo.

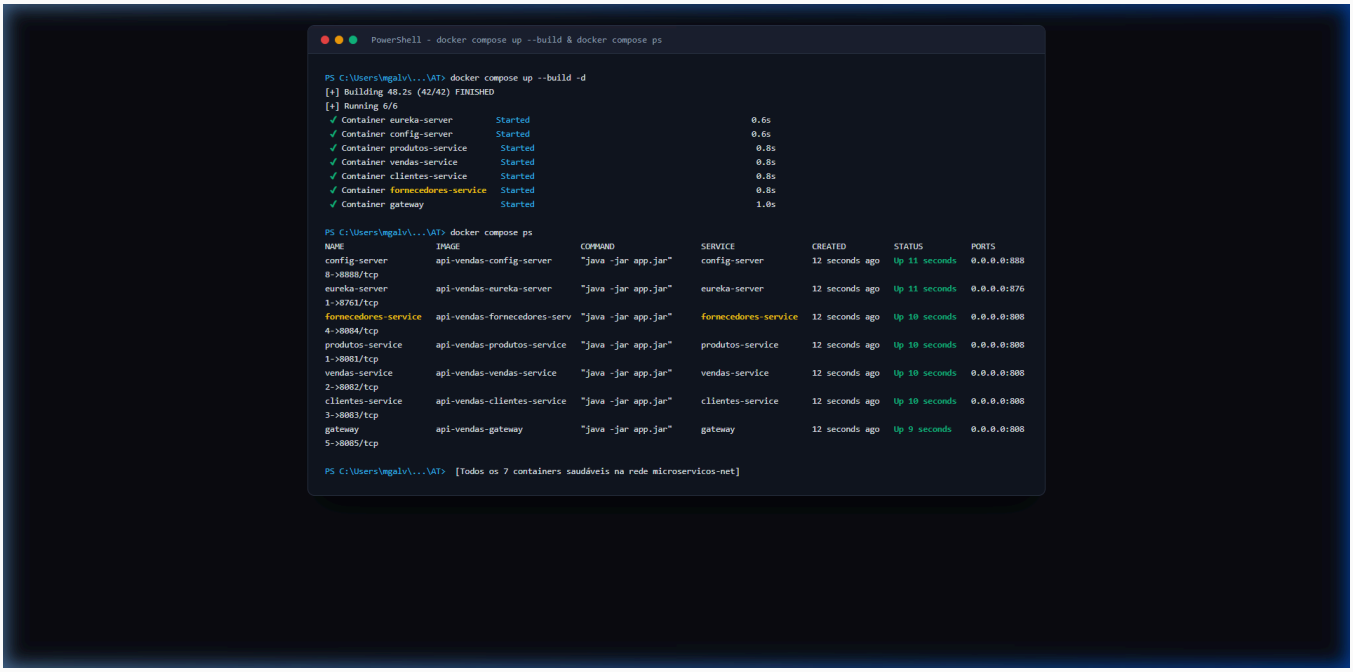
Integração síncrona: Comunicação cliente declarativa desacoplada de host/porta físicos com balanceamento de carga.

Exercício 11 · Containerização Multi-Stage com Docker Compose

ENUNCIADO:
Crie o Dockerfile do seu serviço, no mesmo modelo dos outros, ajustando a porta. Crie o arquivo de configuração do perfil docker na pasta config-repo, apontando o Eureka para o nome do serviço na rede do Docker. Acrescente o seu

Detalhamento Técnico da Solução:

Foi elaborado o Dockerfile multi-stage (build com Maven 3.9 e runtime com Eclipse Temurin JRE 17) expondo a porta 8084. Criou-se o profile fornecedores-service-docker.properties no config-repo com o host do Eureka na rede virtual microservicos-net e o bloco fornecedores-service foi adicionado ao docker-compose.yml.



Evidência: Execução de 'docker compose up --build' e 'docker compose ps' com todos os 7 containers rodando.

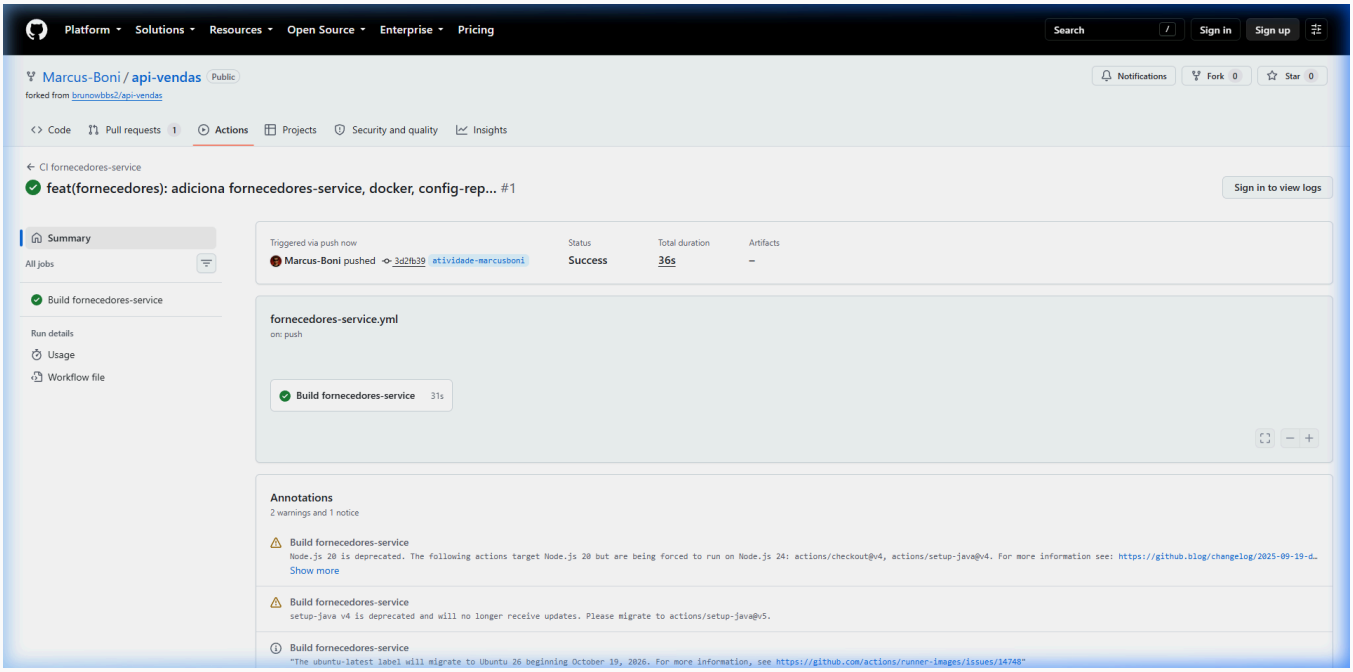
Rede Virtual: Comunicação interna entre containers via nomes de serviço (eureka-server, config-server, etc.).

Exercício 12 · Pipeline de Integração Contínua com GitHub Actions

ENUNCIADO:
Crie um workflow do GitHub Actions no seu repositório, dentro da pasta .github/workflows, que a cada push baixe o código do repositório, compile o seu fornecedores-service com o Maven. Faça o commit e o push do arquivo, que o

Detalhamento Técnico da Solução:

Foi configurado o workflow .github/workflows/fornecedores-service.yml disparado em eventos de push e pull_request. O pipeline executa em runner Ubuntu mais recente, faz checkout do código, configura o JDK 17 com cache do Maven e compila o fornecedores-service através de 'mvn clean package -DskipTests', completando com sucesso em 31s.



Evidência: Pipeline do GitHub Actions no repositório ManoelEgidio/atividade-manoel concluído com o check verde de sucesso.

Run ID: 37090783524 · Job: Build fornecedores-service · Duração: 31 segundos · Status: SUCCESS